



LV12 string 클래스 구현해보기 (Implementing std::string)

소유자	암암코딩
태그	
생성 일시	2024년 3월 2일 오후 7:19

<https://youtu.be/evQbRsNbjFk?feature=shared>

<https://youtu.be/UL7xK18VoHI?feature=shared>

이번 시간에 배울 내용

이번 강의에서는 **C++의 std::string** 클래스를 직접 만들어 봅니다. 지금까지 배운 클래스, 생성자, 연산자 오버로딩, 동적 메모리 할당 등 모든 개념을 종합적으로 활용하는 실전 프로젝트입니다.

왜 string 클래스를 만들어 볼까요?

- C 스타일 문자열(`char*`)의 불편함을 해결하기 위해
- 실제 `std::string`이 어떻게 동작하는지 이해하기 위해
- 객체지향 프로그래밍의 실전 감각을 익히기 위해

사전 지식: C 스타일 문자열의 문제점

C 스타일 문자열이란?

```
char str[] = "Hello"; // 문자 배열로 문자열 표현
char* ptr = "World"; // 포인터로 문자열 표현
```

왜 불편할까?

1. 크기가 고정되어 있어요

```
char str[10] = "Hello";
// "Hello"는 5글자인데 10칸을 미리 할당해야 함
// 나중에 더 긴 문자열을 넣으려면? 불가능!
```

2. 문자열 연결이 복잡해요

```
char str1[20] = "Hello";
char str2[] = " World";
strcat(str1, str2); // 복잡한 함수 사용 필요
```

3. 길이 계산도 직접 해야 해요

```
int len = strlen(str); // 함수 호출 필요
```

std::string은 이런 문제를 해결합니다

```
std::string str = "Hello";
str += " World"; // 간단한 연결!
int len = str.length(); // 길이도 간단하게!
```

string 클래스의 설계도

필요한 재료 (멤버 변수)

우리가 만들 string 클래스는 세 가지 정보를 저장해야 합니다:

```
private:
    char* mStr;           // 1 실제 문자열 데이터
    int mSize;            // 2 현재 문자열 길이
    int mCapacity;        // 3 할당된 메모리 크기
```

왜 이 세 가지가 필요할까요?

mStr (실제 문자열)

문자열 데이터를 저장하는 곳입니다. **동적 할당**으로 만들어서 크기를 자유롭게 조절할 수 있습니다.

```
// 예시
mStr = new char[15]; // 15칸짜리 공간 확보
// mStr이 가리키는 메모리: [H][e][l][l][o][\0][ ][ ][ ][ ][ ]...
```

mSize (현재 길이)

지금 문자열이 몇 글자인지 기억합니다. 매번 `strlen()` 을 호출하지 않아도 됩니다!

```
// "Hello"를 저장했다면
mSize = 5; // 5글자
```

mCapacity (용량)

실제로 할당한 메모리가 몇 칸인지 기억합니다. **여유 공간을 미리 확보**해서 나중에 문자열을 추가할 때 매번 새로 할당하지 않아도 됩니다.

```
// "Hello"(5글자)를 위해 15칸을 할당했다면
mCapacity = 15; // 나머지 10칸은 여유 공간
```

왜 여유 공간이 필요할까요?

비유: 가방을 살 때 딱 맞는 크기보다 여유있게 사는 이유와 같습니다.

```
// 여유 공간이 없다면:
string str = "Hello"; // 5칸 할당
str += " ";           // 1칸 더 필요 → 새로 6칸 할당
```

```

str += "W";           // 1칸 더 필요 → 새로 7칸 할당
str += "o";           // 1칸 더 필요 → 새로 8칸 할당
// 매번 새로 할당하면 느려요!

// 여유 공간이 있다면:
string str = "Hello"; // 15칸 할당 (여유 10칸)
str += " World";       // 여유 공간 사용 → 새로 할당 안 함!

```

1단계: 생성자 만들기

생성자가 하는 일

생성자는 객체가 처음 만들어질 때 **초기화** 작업을 합니다.

```

ya::string str("Hello"); // 이 순간 생성자 호출!

```

생성자 코드

```

string(const char* str)
{
    mSize = strlen(str);           // 1 길이 계산

    mCapacity = (mSize * 2) + (mSize / 2); // 2 여유 공간
    계산

    mStr = new char[mCapacity];     // 3 메모리 할
    당
    memset(mStr, 0, mCapacity);    // 4 메모리 초
    기화
    memcpy(mStr, str, mSize + 1);  // 5 문자열 복
    사
}

```

단계별 자세한 설명

길이 계산

```
mSize = strlen(str);
```

- `strlen()` 함수는 `\0` (널 문자) 전까지의 길이를 셉니다
- "Hello"라면 → `mSize = 5`

여유 공간 계산

```
mCapacity = (mSize * 2) + (mSize / 2);
```

왜 이렇게 계산할까요?

현재 크기의 **2.5배**를 할당합니다. 충분한 여유 공간을 확보하기 위해서입니다.

```
// 예시: "Hello" (5글자)
mCapacity = (5 * 2) + (5 / 2)
           = 10 + 2
           = 12칸

// 예시: "Hello World" (11글자)
mCapacity = (11 * 2) + (11 / 2)
           = 22 + 5
           = 27칸
```

메모리 할당

```
mStr = new char[mCapacity];
```

- `new` 로 동적 메모리를 할당합니다
- `mCapacity` 크기만큼 공간을 확보합니다

메모리 그림:

```
할당 전:
mStr = nullptr (아무것도 가리키지 않음)

할당 후 (mCapacity = 12):
```

```
mStr → [ ][ ][ ][ ][ ][ ][ ][ ][ ][ ][ ][ ]
         0  1  2  3  4  5  6  7  8  9  10 11
```

메모리 초기화

```
memset(mStr, 0, mCapacity);
```

- 할당한 메모리를 모두 0으로 채웁니다
- **왜 필요한가요?** 새로 할당한 메모리에는 쓰레기 값이 들어있을 수 있어서 깨끗하게 정리합니다

초기화 전:

```
mStr → [?][?][?][?][?][?][?][?][?][?][?][?]
        (쓰레기 값)
```

초기화 후:

```
mStr → [0][0][0][0][0][0][0][0][0][0][0][0]
```

문자열 복사

```
memcpy(mStr, str, mSize + 1);
```

- 매개변수로 받은 문자열을 우리 메모리에 복사합니다
- **왜 mSize + 1 일까요?** 널 문자(\0)까지 복사하기 위해서입니다!

// "Hello" 복사 시

```
str  : [H][e][l][l][o][\0]
```

↓ ↓ ↓ ↓ ↓ ↓

```
mStr : [H][e][l][l][o][\0][0][0][0][0][0][0]
```

생성자 전체 동작 예시

```
ya::string str("Hello");
```

1단계: `mSize = strlen("Hello") = 5`

2단계: `mCapacity = (5 * 2) + (5 / 2) = 12`

3단계: `mStr = new char[12]`
→ 12칸 메모리 할당

4단계: `memset(mStr, 0, 12)`
→ `[0][0][0][0][0][0][0][0][0][0][0][0]`

5단계: `memcpy(mStr, "Hello", 6)`
→ `[H][e][l][l][o][\0][0][0][0][0][0][0]`

최종 상태:

`mStr: [H][e][l][l][o][\0][0][0][0][0][0][0]`

`mSize: 5`

`mCapacity: 12`

2단계: 소멸자 만들기

소멸자가 하는 일

객체가 사라질 때 **메모리 정리**를 합니다. 생성자에서 `new` 로 할당한 메모리를 `delete` 로 해제해야 **메모리 누수**를 방지할 수 있습니다.

소멸자 코드

```
~string()  
{  
    delete[] mStr;          // 1 메모리 해제  
    mStr = nullptr;         // 2 포인터 초기화  
}
```

왜 필요한가요?

```
{
    ya::string str("Hello"); // 생성자: new로 메모리 할당
    // str 사용...
} // 이 중괄호를 벗어나면 str 객체 소멸
    // 소멸자: delete로 메모리 해제 필요!
```

메모리 누수 방지:

```
// 소멸자가 없다면:
void test()
{
    ya::string str1("Hello"); // 메모리 할당
    ya::string str2("World"); // 메모리 할당
    ya::string str3("!!!!"); // 메모리 할당
} // 함수 종료 → 메모리가 해제 안 됨 → 메모리 누수!

// 소멸자가 있다면:
void test()
{
    ya::string str1("Hello"); // 메모리 할당
    ya::string str2("World"); // 메모리 할당
    ya::string str3("!!!!"); // 메모리 할당
} // 함수 종료 → 소멸자 자동 호출 → 메모리 해제 완료!
```

delete[] vs delete

```
delete[] mStr; //  배열 할당(new[])에는 delete[] 사용
delete mStr;    //  잘못된 사용! 일부만 해제됨
```

3단계: += 연산자 만들기

+= 연산자가 하는 일

기존 문자열 뒤에 새로운 문자열을 추가합니다.


```
ya::string str("Hello");
str += " World"; // "Hello World"가 됨
```

+= 연산자 코드

```
void operator+= (const char* str)
{
    int len = strlen(str);           // 1 추가할 문자열
    길이
    int newSize = mSize + len;       // 2 새로운 전체 길
    이

    // 3 용량 체크 및 재할당
    if (newSize >= mCapacity)
    {
        mCapacity = (newSize * 2) + (newSize / 2);
        char* newStr = new char[mCapacity];
        memset(newStr, 0, mCapacity);
        memcpy(newStr, mStr, mSize);

        delete[] mStr;
        mStr = nullptr;

        mStr = newStr;
    }

    // 4 문자열 추가
    memcpy(mStr + mSize, str, len + 1);
    mSize = newSize;
}
```

단계별 자세한 설명

추가할 문자열 길이 계산

```
int len = strlen(str);
```

```
// 예시
str += " World"; // 추가할 문자열
len = strlen(" World") = 6
```

새로운 전체 길이 계산

```
int newSize = mSize + len;
```

```
// 예시: "Hello"에 " World" 추가
mSize = 5 // 기존 "Hello"
len = 6 // 추가할 " World"
newSize = 5 + 6 = 11
```

용량 체크 및 재할당 (중요!)

용량이 충분한 경우:

```
현재 상태:
mStr: [H][e][l][l][o][\0][0][0][0][0][0][0]
mSize: 5
mCapacity: 12

" World" (6글자) 추가 시:
newSize = 11
11 < 12 (용량 충분!)
→ 재할당 없이 바로 추가
```

용량이 부족한 경우:

```
현재 상태:
mStr: [H][e][l][l][o][\0][0][0]
mSize: 5
mCapacity: 8

" World !!!!!" (12글자) 추가 시:
newSize = 17
```

17 >= 8 (용량 부족!)

→ 재할당 필요!

재할당 과정 상세 설명

```
// 3-1. 새로운 용량 계산
mCapacity = (newSize * 2) + (newSize / 2);
// newSize = 17이면
// mCapacity = (17 * 2) + (17 / 2) = 34 + 8 = 42

// 3-2. 새 메모리 할당
char* newStr = new char[mCapacity];
// newStr → [0][0][0]...[0] (42칸)

// 3-3. 새 메모리 초기화
memset(newStr, 0, mCapacity);

// 3-4. 기존 데이터 복사
memcpy(newStr, mStr, mSize);
// mStr:   [H][e][l][l][o][\0][0][0]
//           ↓  ↓  ↓  ↓  ↓  ↓
// newStr: [H][e][l][l][o][0][0][0]...[0] (42칸)

// 3-5. 기존 메모리 해제
delete[] mStr;
mStr = nullptr;

// 3-6. 새 메모리로 교체
mStr = newStr;
```

그림으로 이해하기:

재할당 전:

mStr → [H][e][l][l][o][\0][0][0] (8칸, 부족함)

재할당 중:

mStr → [H][e][l][l][o][\0][0][0] (기존)

newStr → [H][e][l][l][o][0][0]...[0] (42칸, 새로 할당)

재할당 후:

mStr → [H][e][l][l][o][0][0]...[0] (42칸)
(기존 8칸 메모리는 delete됨)

문자열 추가

```
memcpy(mStr + mSize, str, len + 1);  
mSize = newSize;
```

포인터 연산 이해하기:

mStr + mSize // 기존 문자열의 끝 위치를 가리킴

// 예시: "Hello"의 경우

mStr → [H][e][l][l][o][\0][0]...[0]

↑

mStr

↑

mStr + mSize (5번 위치, \0가 있는 곳)

복사 과정:

```
// " World" 추가 시  
memcpy(mStr + mSize, " World", len + 1);  
//      ↑ 복사 시작 위치 (5번 인덱스)  
//                               ↑ 6글자 + \0 = 7바이트
```

복사 전:

mStr: [H][e][l][l][o][\0][0][0][0][0][0][0]

↑ 여기부터 복사

복사 후:

mStr: [H][e][l][l][o][][W][o][r][l][d][\0]

mSize: 11

전체 동작 예시

```
ya::string str("Hello");    // 초기 상태
str += " World";            // 문자열 추가
```

초기 상태:

```
mStr: [H][e][l][l][o][\0][0][0][0][0][0][0]
mSize: 5
mCapacity: 12
```

`+= " World"` 실행:

1단계: `len = 6, newSize = 11`

2단계: 용량 체크

`newSize (11) < mCapacity (12) → 용량 충분!`

3단계: 문자열 추가

```
memcpy(mStr + 5, " World", 7)
```

최종 상태:

```
mStr: [H][e][l][l][o][ ][W][o][r][l][d][\0]
mSize: 11
mCapacity: 12
```

4단계: [] 연산자 만들기

[] 연산자가 하는 일

배열처럼 인덱스로 문자에 접근할 수 있게 합니다.

```
ya::string str("Hello");
cout << str[0];    // 'H' 출력 (읽기)
str[1] = 'a';      // "Hallo"로 변경 (쓰기)
```

[] 연산자 코드

```
char& operator[] (int index)
{
    return mStr[index];
}
```

왜 참조(&)를 반환할까요?

참조 반환의 의미:

```
char& operator[] (int index) // ✅ 참조 반환
{
    return mStr[index];
}

// 이렇게 동작:
str[0] = 'h';
// ↓ 실제로는
mStr[0] = 'h'; // 직접 수정 가능!
```

참조가 아니면 어떻게 될까요?

```
char operator[] (int index) // ❌ 값 반환
{
    return mStr[index];
}

// 이렇게 동작:
str[0] = 'h';
// ↓ 실제로는
char temp = mStr[0]; // 복사본 생성
temp = 'h';          // 복사본만 수정됨
// mStr[0]은 그대로! 원본 수정 안 됨!
```

읽기와 쓰기 모두 가능

```
ya::string str("Hello");
```

```
// 읽기: 값을 가져옴
cout << str[0];           // 'H' 출력
char c = str[1];          // c = 'e'

// 쓰기: 값을 변경
str[0] = 'h';              // "hello"로 변경
str[4] = 'p';              // "hellp"로 변경
```

작동 원리 상세 설명

```
ya::string str("Hello");
str[1] = 'a';
```

1단계: operator[] 호출
 str[1]
 → operator[](1) 호출
 → return mStr[1]; (참조 반환)

2단계: 반환된 참조에 값 대입
 mStr[1] = 'a';

메모리 변화:
 변경 전: [H][e][l][l][o][\0]
 ↓
 변경 후: [H][a][l][l][o][\0]

5단계: 유틸리티 함수들

Size() 함수

현재 문자열의 길이를 반환합니다.

```
int Size() const
{
```

```
    return mSize;
}
```

const가 붙은 이유

```
int Size() const // ← 이 함수는 멤버 변수를 수정하지 않음을 보장
```

- `const` 함수는 읽기 전용 함수입니다
- 멤버 변수를 변경할 수 없습니다
- 안전성을 보장합니다

```
ya::string str("Hello");
int len = str.Size(); // ✓ 안전하게 길이만 읽어옴
```

c_str() 함수

C 스타일 문자열 포인터를 반환합니다. `cout` 같은 C 함수와 호환되기 위해 필요합니다.

```
const char* c_str() const
{
    return mStr;
}
```

왜 필요한가요?

```
ya::string str("Hello");

// cout은 char* 타입을 출력할 수 있음
cout << str.c_str(); // ✓ "Hello" 출력

// 직접 출력하면?
cout << str;         // ✗ 에러! (연산자가 없음)
```

const가 두 번 붙은 이유


```
const char* c_str() const
//      ↑                ↑
//      |                함수가 멤버를 수정 안 함
//      반환된 문자열을 수정하면 안 됨
```

```
ya::string str("Hello");
const char* ptr = str.c_str();

ptr[0] = 'h'; // ❌ 에러! const char*는 수정 불가
```

전체 코드와 사용 예시

완성된 ya::string 클래스

```
#include <iostream>
#include <cstring> // strlen, memcpy, memset

namespace ya
{
    class string
    {
    public:
        // 생성자
        string(const char* str)
        {
            mSize = strlen(str);
            // 넉넉하게 할당 (기본 크기보다 조금 더 크게)
            mCapacity = (mSize * 2) + (mSize / 2);
            if (mCapacity == 0) mCapacity = 1; // 빈 문자열
            대비

            mStr = new char[mCapacity];
            memset(mStr, 0, mCapacity);
            memcpy(mStr, str, mSize + 1); // NULL 문자 포함
```

복사

```
}

// 소멸자
~string()
{
    if (mStr != nullptr)
    {
        delete[] mStr;
        mStr = nullptr;
    }
}

// += 연산자
void operator+= (const char* str)
{
    int len = strlen(str);
    int newSize = mSize + len;

    // 용량이 부족할 경우 재할당
    if (newSize >= mCapacity)
    {
        // 용량 늘리기 (기존 로직 유지)
        mCapacity = (newSize * 2) + (newSize / 2);

        char* newStr = new char[mCapacity];
        memset(newStr, 0, mCapacity);

        // 1. 기존 데이터 복사 (새 공간으로)
        memcpy(newStr, mStr, mSize);

        // 2. 기존 메모리 해제
        delete[] mStr;

        // 3. 포인터 교체 (중요!)
        mStr = newStr;
    }
}
```

```

        // 4. 문자열 이어 붙이기 (기존 끝부분부터, NULL 문자
포함)
        memcpy(mStr + mSize, str, len + 1);

        // 5. 사이즈 갱신
        mSize = newSize;
    }

    // [] 연산자: 인덱스로 문자 접근
    char& operator[] (int index)
    {
        return mStr[index];
    }

    // Size 함수
    int Size() const
    {
        return mSize;
    }

    // c_str 함수 (문자열 주소 반환)
    const char* c_str() const
    {
        return mStr;
    }

private:
    char* mStr;
    int mSize;
    int mCapacity;
};

int main()
{
    // 테스트 코드
    ya::string s("Hello");
    std::cout << "처음: " << s.c_str() << std::endl;

```

```

s += " World!";
std::cout << "변경 후: " << s.c_str() << std::endl;
std::cout << "길이: " << s.Size() << std::endl;
std::cout << "인덱스 [1]: " << s[1] << std::endl;

return 0;
}

```

실습 코드

```

int main()
{
    // std::string 사용법 비교
    std::string stdStr("Hello");
    stdStr += " World";
    cout << "std::string: " << stdStr << endl;
    cout << "4번째 문자: " << stdStr[4] << endl;

    cout << "—————" << endl;

    // 우리가 만든 ya::string 사용
    ya::string myStr("Hello");
    cout << "초기 문자열: " << myStr.c_str() << endl;
    cout << "길이: " << myStr.Size() << endl;

    // 문자열 추가
    myStr += " World";
    cout << "\n문자열 추가 후: " << myStr.c_str() << endl;
    cout << "길이: " << myStr.Size() << endl;

    // 개별 문자 변경
    myStr[0] = 'h'; // 'H' → 'h'
    cout << "\n첫 글자 변경 후: " << myStr.c_str() << endl;

    // 인덱스 접근

```

```
cout << "4번째 문자: " << myStr[4] << endl;

return 0;
}
```

실행 결과

```
std::string: Hello World
4번째 문자: o
```

```
초기 문자열: Hello
길이: 5
```

문자열 추가 후: Hello World

길이: 11

첫 글자 변경 후: hello World

4번째 문자: o

핵심 개념 정리

동적 메모리 관리

왜 동적 할당을 사용하나요?

```
// ❌ 정적 할당: 크기가 고정됨
char str[10]; // 최대 9글자만 저장 가능
```

```
mCapacity: [H][e][l][l][o][\0][ ][ ][ ][ ][ ][ ][ ][ ][ ] (12칸 할당)
mSize: 5   ↑—————↑ (실제 사용 5칸)
mStr: 포인터가 첫 번째 위치를 가리킴
```

```
// ✅ 동적 할당: 필요한 만큼 확장 가능
char* str = new char[size]; // 필요한 크기만큼 할당
```

메모리 관리 3단계

```
// 1단계: 할당
mStr = new char[mCapacity];

// 2단계: 사용
memcpy(mStr, "Hello", 6);

// 3단계: 해제 (소멸자에서)
delete[] mStr;
```

2. 연산자 오버로딩

왜 연산자를 오버로딩하나요?

직관적인 코드 작성:

```
// ❌ 연산자 오버로딩 없이
str.append(" World");
char c = str.at(0);

// ✅ 연산자 오버로딩으로
str += " World";      // 더 직관적!
char c = str[0];      // 더 간단!
```

구현한 연산자들

- +=: 문자열 추가
- []: 인덱스 접근

3. 참조 반환의 중요성

```
// ✅ 참조 반환: 원본 수정 가능
char& operator[] (int index) { return mStr[index]; }
str[0] = 'h'; // 원본이 바뀜

// ❌ 값 반환: 복사본만 수정됨
```

```
char operator[] (int index) { return mStr[index]; }  
str[0] = 'h'; // 원본은 그대로
```

4. const의 의미

```
int Size() const // 이 함수는 멤버를 수정하지 않음
```

- 읽기 전용 함수임을 보장
- 실수로 값을 변경하는 것을 방지
- 코드의 안전성 향상

5. 메모리 효율성

Capacity 전략

```
mCapacity = (mSize * 2) + (mSize / 2); // 2.5배 할당
```

왜 여유 공간을 남기나요?

여유 공간 없이:
문자 추가할 때마다 재할당 → 느림!

여유 공간 있음:
여유 공간에 추가 → 빠름!
재할당 횟수 감소 → 효율적!

실습 과제

기본 과제: string 클래스 마스터하기

1단계: 따라 치기 (3회)

- 1회: 코드 보면서 천천히 타이핑
- 2회: 코드 보면서 디버깅하며 타이핑

3회: 코드 안 보고 main만 참고하여 구현

2단계: 완전히 이해했는지 확인

다음 질문에 답할 수 있나요?

1. **mCapacity**를 왜 **mSize**보다 크게 할당하나요?
2. 소멸자에서 `delete[]`를 왜 꼭 해야 하나요?
3. `[]` 연산자에서 왜 참조(&)를 반환하나요?
4. `+=` 연산자에서 언제 재할당이 일어나나요?

응용 과제: 기능 추가하기

난이도 ★: 간단한 함수 추가

```
// 1. 문자열이 비어있는지 확인
bool IsEmpty() const
{

}

// 2. 문자열 비우기
void Clear()
{

}

// 사용 예시
ya::string str("Hello");
if (!str.IsEmpty())
    str.Clear();
```

난이도 ★★: 비교 연산자

```
// == 연산자 구현
bool operator==(const char* str)
{
```



```

        // return true ? false;
    }

    // 사용 예시
    ya::string str("Hello");
    if (str == "Hello")
        cout << "같습니다!" << endl;

```

난이도 ★★★★★: 복사 생성자

```

// 다른 string 객체로부터 복사
string(const string& other)
{
    mSize = other.mSize;
    mCapacity = other.mCapacity;

    mStr = new char[mCapacity];
    memset(mStr, 0, mCapacity);
    memcpy(mStr, other.mStr, mSize + 1);
}

// 사용 예시
ya::string str1("Hello");
ya::string str2(str1); // str1을 복사

```

도전 과제: std::vector 구현하기

std::vector란?

크기가 자동으로 늘어나는 배열입니다.

```

#include <vector>

std::vector<int> vec;
vec.resize(100);
vec.push_back(1); // 요소 추가
vec.push_back(2);
vec.push_back(3);

```

```
cout << vec[0];    // 1 출력
cout << vec.size(); // 3 출력
```

구현할 기능들

```
template<typename T>
class vector
{
public:
    vector();           // 생성자
    ~vector();          // 소멸자

    void push_back(T value); // 요소 추가
    void clear();           // 모든 요소 제거
    void resize(int newSize); // 크기 변경(사용 가능 공간)
    void reserve(int size);  // 실제 메모리 할당 크기

    T& operator[](int index); // 인덱스 접근
    int size() const;         // 크기 반환

private:
    T* mData;           // 실제 데이터 배열
    int mSize;          // 현재 요소 개수
    int mCapacity;      // 할당된 용량
};
```

힌트

string 클래스와 비슷하지만 다른 점:

- `char` 가 아닌 **템플릿(T)** 사용
- 문자열 복사(`memcpy`) 대신 **반복문으로 복사**
- `strlen` 대신 **size 변수로 관리**

```
// push_back 구현 힌트
void push_back(T value)
{
```

```

// 1. 용량 체크
if (mSize >= mCapacity)
{
    // 재할당 (string과 비슷)
}

// 2. 요소 추가
mData[mSize] = value;
mSize++;
}

```

이번 강의를 통해 배운 것들

1. 객체지향의 실전 적용

- 캡슐화: 내부 구현(`mStr`, `mSize`, `mCapacity`)을 `private` 으로 숨김
- 인터페이스: 사용하기 쉬운 함수들(`+=`, `[]`, `Size()`) 제공
- 생성자/소멸자: 자동 초기화와 정리

2. 메모리 관리의 중요성

- 동적 할당: 필요한 만큼만 메모리 사용
- 메모리 누수 방지: 소멸자에서 확실하게 해제
- 재할당 전략: 여유 공간으로 효율성 향상

3. 연산자 오버로딩의 활용

- 직관적인 코드: `str += "abc"` 처럼 자연스러운 표현
- 타입 안전성: C 스타일보다 안전한 사용

4. STL 라이브러리의 이해

- `std::string` 이 어떻게 동작하는지 이해
- 다른 STL 컨테이너(`vector`, `list` 등)도 비슷한 원리로 동작

다음 단계

더 배워야 할 것들

1. 복사 생성자와 대입 연산자 (Deep Copy)
2. **Move** 문법 (C++11)
3. 템플릿을 활용한 제네릭 프로그래밍
4. 예외 처리 (메모리 할당 실패 등)

실전 적용

- 기존 문제들을 `std::string`, `std::vector` 로 다시 풀기
- 직접 만든 클래스로 실제 프로그램 작성해보기
- 성능 비교: 직접 구현 vs STL

자주 묻는 질문 (FAQ)

Q1. 왜 mCapacity를 mSize보다 크게 만드나요?

A. 매번 문자열을 추가할 때마다 재할당하면 느리기 때문입니다.

```
// 여유 공간 없이 (비효율적)
"H" → 재할당
"He" → 재할당
"He!" → 재할당
// 매번 재할당!
```

```
// 여유 공간 있음 (효율적)
"H" → 재할당 (12칸 확보)
"He" → 여유 공간 사용
"He!" → 여유 공간 사용
// 한 번만 재할당!
```

Q2. memcpy와 strcpy의 차이는?

A.

- **memcpy**: 정확히 N바이트를 복사

- **strcpy**: `\0`을 만날 때까지 복사

```
char src[] = "Hello";

// memcpy: 정확히 6바이트 복사
memcpy(dest, src, 6);

// strcpy: \0까지 자동 복사
strcpy(dest, src);
```

Q3. delete와 delete[]의 차이는?

A.

- **delete**: 단일 객체 해제
- **delete[]**: 배열 해제

```
int* a = new int;           // 단일 할당
delete a;                   // 단일 해제

int* arr = new int[10];     // 배열 할당
delete[] arr;               // 배열 해제
```

Q4. const char* c_str() const에서 const가 2개인 이유는?

A.

```
const char* c_str() const
//   ↑           ↑
//   |           2번째: 함수가 멤버를 수정 안 함
//   1번째: 반환된 포인터로 문자열 수정 안 함
```

Q5. 참조 반환이 위험하지 않나요?

A. 멤버 변수를 참조 반환하는 것은 안전합니다. 객체가 살아있는 동안 멤버도 유효하기 때문입니다.

```

char& operator[](int index)
{
    return mStr[index]; // ✔️ 안전 (객체가 살아있음)
}

// 위험한 경우:
char& GetChar()
{
    char c = 'A';
    return c; // ❌ 위험! 지역 변수 반환
}

```

수고하셨습니다!

string 클래스를 직접 구현하면서 객체지향 프로그래밍의 핵심을 모두 경험해보았습니다.
이제 STL을 사용할 때 내부에서 어떤 일이 일어나는지 이해할 수 있을 것입니다!

다음 강의에서는 템플릿을 더 깊이 다루고, 제네릭 프로그래밍을 배워보겠습니다.